



SIR M VISVESVARAYA INSTITUTE OF TECHNOLOGY

*(Affiliated to VTU, Recognized by AICTE and Accredited by NBA, NAAC
and an ISO 9001-2008 Certified Institution)*
Bengaluru – 562157



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(IoT & Cyber Security including Block Chain Technology)

DATA STRUCTURES LABORATORY

1BCSL306 - III Semester B.E

(Academic Year 2026-2027)

Compiled and Prepared by:
Mr. Madhusudhana V
Assistant Professor,
Dept. of IOT

Reviewed By:
Dr. Savita Choudhary,
Professor & Head,
Dept. of IOT

Department Vision and Mission

VISION

To build a center for imparting quality technical education and carrying out research activity to meet the current and future challenges in the domain of Computer Science and Engineering.

MISSION

- The Computer Science and Engineering department strives for excellence in teaching, applying, promoting and imparting knowledge through comprehensive academic curricula.
- Train students to effectively apply the knowledge to solve real-world problems, thus enhance their potential for life-long high-quality career and give them a competitive advantage in the ever-changing and fast paced computing.
- Prepare students to demonstrate a sense of societal and ethical responsibilities in their professional endeavors.
- Creating amongst students and faculty a collaborative environment open to the free exchange of ideas, which leads to research activity and fuels innovation thinking.

PROGRAM OUTCOMES

PO's	PO Description
PO1	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
PO3	Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4	Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
PO5	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO6	The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
PO7	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
PO8	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences
PO10	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
PO11	Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

PROGRAM SPECIFIC OUTCOMES

PSOs	PSO Description
PSO1	An ability to design and analyze algorithms by applying theoretical concepts to build complex and computer- based systems in the domain of System Software, Computer Networks & Security, Web technologies, Data Science and Analytics.
PSO2	Be able to develop various software solutions by applying the techniques of Data Base Management, Complex Mathematical Models, Software Engineering practices and Machine Learning with Artificial Intelligence.

INDEX

Course code	1BCSL306	SCHEME	2025
Number of Contact Hours/Weeks	2	CIE Marks	50
Total Number of Lab Contact Hours	28	SEE Marks	50
Credits	01	EXAM HOURS	03

Sl.NO	Experiments	Pg No.
PART-A FIXED SET OF EXPERIMENTS		
1	1. Develop a Program in C for the following: a. Define a structure Book with the following fields: Book_ID, Title, Author, Price, Availability Status (Available/Issued) b. Dynamically allocate memory to store the details of N books, where N is entered by the user. c. Implement the following functions: i. create() – To allocate memory and read the details of N books and store them in the allocated memory. ii. display() – To display the details of all books available in the library. iii. search() – To search for a book using Book ID and display the book details if found. iv. issueBook() – To issue a book by updating its availability status and display an appropriate message. v. returnBook() – To return an issued book by updating its availability status and display an appropriate message. d. Provide a menu-driven interface with the following options: i. Add Book Records ii. Display All Book Records iii. Search Book by Book ID iv. Issue a Book v. Return a Book vi. Exit	1-7
2	Develop a menu driven Program in C for the following operations on STACK of Integers (Array Implementation of Stack with maximum size MAX) a. Push an Element on to Stack b. Pop an Element from Stack c. Demonstrate how Stack can be used to check Palindrome d. Demonstrate Overflow and Underflow situations on Stack e. Display the status of Stack f. Exit Support the program with appropriate functions for each of the above operations	8-13
3	Develop a program in C for simulating a Printer Queue using Queue data structure with the following operations: a. Add a print job to the Printer Queue. b. Process (delete) a print job from the Printer Queue. c. Display all pending print jobs in the Queue. d. Display the number of print jobs currently waiting in the Queue. e. Demonstrate Queue overflow and underflow conditions.	14-19

	f. Exit.	
4	Develop a menu-driven Program in C for the following operations on a Singly Linked List of Student Records: a. Create a linked list using front insertion. b. Display all student records and count the number of nodes. c. Insert and delete nodes at the beginning and end of the list. d. Search for a student record using USN	20-31
5	Develop a Program in C for the following: a. Create a Binary Tree by inserting integer elements entered by the user in level order. b. Implement the following functions: i. createTree() – Create a binary tree using dynamic memory allocation. ii. preorder() – Display the elements using Preorder Traversal. iii. inorder() – Display the elements using Inorder Traversal. iv. postorder() – Display the elements using Postorder Traversal. c. Provide a menu-driven interface with the following options: i. Create Binary Tree ii. Display Preorder Traversal iii. Display Inorder Traversal iv. Display Postorder Traversal v. Exit	31-40
6	Develop a Program in C for the following operations on Graph(G) of Cities a. Create an Adjacency Matrix representing the graph of N cities b. Print all the nodes reachable from a given starting node in a digraph using DFS/BFS method.	40-47
PART – B OPEN ENDED EXPERIMENTS		
1	Develop a C program to: a. Read two sparse matrices of the same order. b. Convert both into 3-tuple representation. c. Perform matrix addition. d. Display the resultant matrix in both normal matrix form and 3-tuple form.	48-54
2	Develop a C-based Expression Conversion tool that uses stack data structure to convert arithmetic expressions from infix notation to postfix notation. The tool should support alphanumeric operands, parentheses, and the operators (+, -, *, /, %, ^). Analyze the correctness of the conversion by testing various valid expressions.	55-59
3	Develop a menu-driven Program in C for the following operations on Circular Queue (integer/floating point data): a. Insert an element into the Circular Queue. b. Delete an element from the Circular Queue. c. Display the contents of the Circular Queue. d. Demonstrate overflow and underflow conditions. e. Exit.	59-65

4	Develop a menu driven Program in C for the following operations on Doubly Linked List (DLL) of Employee Data with the fields: SSN, Name, Dept, Designation, Sal, PhNo etc. - Create a DLL of N Employees Data by using end insertion. - Display the status of DLL and count the number of nodes in it - Perform Insertion and Deletion at End of DLL - Perform Insertion and Deletion at Front of DLL - Demonstrate how this DLL can be used as Double Ended Queue. - Exit	65-78
5	Develop a menu driven Program in C for the following operations on Binary Search Tree (BST) of Integers. - Create a BST of N Integers - Traverse the BST in Inorder, Preorder and Post Order - Search the BST for a given element (KEY) and report the appropriate message - Exit	78-86
6	Develop a Program in C for an Employee Record Management System using Hashing with the following operations: - Create a hash table to store employee records using Employee ID as the key. - Insert employee records into the hash table using the Hash function $H: K \rightarrow L$ as $H(K)=K \bmod m$ - Resolve collisions using Linear Probing. - Search for an employee record using a given Employee ID and display the details. - Display the contents of the hash table along with the corresponding memory locations.	86-95

PART-A

Exp. No. 1

1. Develop a Program in C for the following:

a. Define a structure **Book** with the following fields: **Book_ID**, **Title**, **Author**, **Price**, **Availability Status** (Available/Issued)

b. Dynamically allocate memory to store the details of N books, where N is entered by the user.

c. Implement the following functions:

i. **create()** – To allocate memory and read the details of N books and store them in the allocated memory.

ii. **display()** – To display the details of all books available in the library.

iii. **search()** – To search for a book using Book ID and display the book details if found.

iv. **issueBook()** – To issue a book by updating its availability status and display an appropriate message.

v. **returnBook()** – To return an issued book by updating its availability status and display an appropriate message.

d. Provide a menu-driven interface with the following options:

i. Add Book Records

ii. Display All Book Records

iii. Search Book by Book ID

iv. Issue a Book

v. Return a Book

vi. Exit

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#define MAX_TITLE 60
```

```
#define MAX_AUTHOR 40
```

```
typedef struct {
    int Book_ID;
    char Title[MAX_TITLE];
    char Author[MAX_AUTHOR];
    float Price;
    int Available; /* 1 = Available, 0 = Issued */
} Book;

static void read_str(const char *prompt, char *buf, int size) {
    printf("%s", prompt);
    if (fgets(buf, size, stdin)) {
        size_t l = strlen(buf);
        if (l && buf[l-1] == '\n') buf[l-1] = '\0';
    } else {
        buf[0] = '\0';
    }
}

/* Add k books to the array (realloc if needed) */
void addBooks(Book **arrp, int *np) {
    int k;
    printf("Enter number of books to add: ");
    if (scanf("%d", &k) != 1 || k <= 0) {
        printf("Invalid number.\n");
        while (getchar() != '\n');
        return;
    }
    while (getchar() != '\n');
    Book *tmp = realloc(*arrp, sizeof(Book) * ((*np) + k));
    if (!tmp) {
        printf("Memory allocation failed.\n");
    }
}
```

```
return;

}

*arrp = tmp;

for (int i = 0; i < k; ++i) {

    Book *b = &((*arrp)[(*np) + i]);

    printf("\nEnter book %d:\n", (*np) + i + 1);

    printf("Book ID: ");

    while (scanf("%d", &b->Book_ID) != 1) {

        printf("Enter integer Book ID: ");

        while (getchar() != '\n');

    }

    while (getchar() != '\n');

    read_str("Title: ", b->Title, MAX_TITLE);

    read_str("Author: ", b->Author, MAX_AUTHOR);

    printf("Price: ");

    while (scanf("%f", &b->Price) != 1) {

        printf("Enter numeric Price: ");

        while (getchar() != '\n');

    }

    while (getchar() != '\n');

    b->Available = 1; /* default available */

}

*np += k;

printf("\n%d books added. Total books: %d\n", k, *np);

}

void display(const Book *arr, int n) {
```

```
if (!arr || n <= 0) { printf("No records to display.\n"); return; }

printf("\n--- Library Records ---\n");

for (int i = 0; i < n; ++i) {

    printf("ID:%d | %s | %s | Rs %.2f | %s\n",

        arr[i].Book_ID, arr[i].Title, arr[i].Author, arr[i].Price,

        arr[i].Available ? "Available" : "Issued");

}

}

int search(const Book *arr, int n, int id) {

    if (!arr) return -1;

    for (int i = 0; i < n; ++i) if (arr[i].Book_ID == id) return i;

    return -1;

}

void issueBook(Book *arr, int n) {

    if (!arr || n <= 0) { printf("No records.\n"); return; }

    int id;

    printf("Enter Book ID to issue: ");

    if (scanf("%d", &id) != 1) { printf("Invalid input.\n"); while (getchar() != '\n'); return; }

    while (getchar() != '\n');

    int idx = search(arr, n, id);

    if (idx == -1) { printf("Book not found.\n"); return; }

    if (arr[idx].Available) {

        arr[idx].Available = 0;

        printf("Book '%s' (ID %d) issued.\n", arr[idx].Title, id);

    } else {

        printf("Book already issued.\n");

    }

}
```

```
void returnBook(Book *arr, int n) {  
    if (!arr || n <= 0) { printf("No records.\n"); return; }  
    int id;  
    printf("Enter Book ID to return: ");  
    if (scanf("%d", &id) != 1) { printf("Invalid input.\n"); while (getchar() != '\n'); return; }  
    while (getchar() != '\n');  
    int idx = search(arr, n, id);  
    if (idx == -1) { printf("Book not found.\n"); return; }  
    if (!arr[idx].Available) {  
        arr[idx].Available = 1;  
        printf("Book '%s' (ID %d) returned.\n", arr[idx].Title, id);  
    } else {  
        printf("Book was not issued.\n");  
    }  
}  
  
int main(void) {  
    Book *books = NULL;  
    int n = 0;  
    int choice;  
    do {  
        printf("\n=== Library Menu ===\n");  
        printf("1. Add Book Records\n");  
        printf("2. Display All Book Records\n");  
        printf("3. Search Book by Book ID\n");  
        printf("4. Issue a Book\n");  
        printf("5. Return a Book\n");  
        printf("6. Exit\n");
```

```
printf("Enter choice: ");

if (scanf("%d", &choice) != 1) { printf("Invalid. Try again.\n"); while (getchar() != '\n'); continue; }

while (getchar() != '\n');

switch (choice) {

    case 1:

        addBooks(&books, &n);

        break;

    case 2:

        display(books, n);

        break;

    case 3: {

        if (n == 0) { printf("No records.\n"); break; }

        int id;

        printf("Enter Book ID to search: ");

        if (scanf("%d", &id) != 1) { printf("Invalid.\n"); while (getchar() != '\n'); break; }

        while (getchar() != '\n');

        int idx = search(books, n, id);

        if (idx == -1) printf("Book not found.\n");

        else printf("Found: ID:%d Title:%s Author:%s Price:%.2f Status:%s\n",

            books[idx].Book_ID, books[idx].Title, books[idx].Author,

            books[idx].Price, books[idx].Available?"Available":"Issued");

        break;

    }

    case 4:

        issueBook(books, n);

        break;

    case 5: returnBook(books, n);
```

```
break;

    case 6:

        printf("Exiting...\n");

        break;

    default:

        printf("Invalid choice.\n");

    }

} while (choice != 6);

free(books);

return 0;

}
```

OUTPUT:

=== Library Menu ===

1. Add Book Records
2. Display All Book Records
3. Search Book by Book ID
4. Issue a Book
5. Return a Book
6. Exit

Enter choice: 1

Enter number of books to add: 1

Entering book 1:

Book ID: 14

Title: DATA STRUCTURES

Author: RAJINI

Price: 780

1 books added. Total books: 1

Exp. No. 2

Develop a menu driven Program in C for the following operations on STACK of Integers (Array Implementation of Stack with maximum size MAX)

- a. Push an Element on to Stack**
- b. Pop an Element from Stack**
- c. Demonstrate how Stack can be used to check Palindrome**
- d. Demonstrate Overflow and Underflow situations on Stack**
- e. Display the status of Stack**
- f. Exit**

Support the program with appropriate functions for each of the above operations

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 5
// Global variables
int s[MAX];
int top = -1;
// Function prototypes
void push(int item);
int pop();
void palindrome();
void display();
void main() {
    int choice, item;
    while (1) {
        printf("\n\n~~~~~Menu~~~~~ : ");
        printf("\n=> 1. Push an Element to Stack and Overflow demo ");
        printf("\n=> 2. Pop an Element from Stack and Underflow demo");
```

```
printf("\n=> 3. Palindrome demo ");
printf("\n=> 4. Display ");
printf("\n=> 5. Exit");
printf("\nEnter your choice: ");
scanf("%d", &choice);
switch (choice) {
    case 1:
        printf("\nEnter an element to be pushed: ");
        scanf("%d", &item);
        push(item);
        break;
    case 2:
        item = pop();
        if (item != -1) {
            printf("\nElement popped is: %d", item);
        }
        break;
    case 3:
        palindrome();
        break;
    case 4:
        display();
        break;
    case 5:
        exit(0)
    default:
        printf("\nPlease enter valid choice ");
        break;
}
```

```
}
```

```
}

// Function to push an element onto the stack

void push(int item) {
    if (top == MAX - 1) {
        printf("\n~~~~~Stack overflow~~~~~");
        return;
    }
    top = top + 1;
    s[top] = item;
}

// Function to pop an element from the stack

int pop() {
    int item;
    if (top == -1) {
        printf("\n~~~~~Stack underflow~~~~~");
        return -1;
    }
    item = s[top];
    top = top - 1;
    return item;
}

// Function to display the stack contents

void display() {
    int i;
    if (top == -1) {
        printf("\n~~~~~Stack is empty~~~~~");
        return;
    }
    printf("\nStack elements are:\n");
    for (i = top; i >= 0; i--) {
```

```
printf("| %d |", s[i]);

}

}

// Function to check if the stack content forms a palindrome
void palindrome() {
    int flag = 1, i;
    if (top == -1) {
        printf("\n~~~~~Stack is empty~~~~~");
        return;
    }
    printf("\nStack content are:\n");
    for (i = top; i >= 0; i--) {
        printf("| %d |", s[i]);
    }
    printf("\nReverse of stack content are:\n");
    for (i = 0; i <= top; i++) {
        printf("| %d |", s[i]);
    }
    for (i = 0; i <= top / 2; i++) {
        if (s[i] != s[top - i]) {
            flag = 0;
            break;
        }
    }
    if (flag == 1) {
        printf("\nIt is a palindrome sequence");
    } else {
        printf("\nIt is not a palindrome sequence");
    }
}
```

OUTPUT:

~~~~~Menu~~~~~ :

- => 1. Push an Element to Stack and Overflow demo
- => 2. Pop an Element from Stack and Underflow demo
- => 3. Palindrome demo
- => 4. Display
- => 5. Exit

Enter your choice: 1

Enter an element to be pushed: 11

~~~~~Menu~~~~~ :

- => 1. Push an Element to Stack and Overflow demo
- => 2. Pop an Element from Stack and Underflow demo
- => 3. Palindrome demo
- => 4. Display
- => 5. Exit

Enter your choice: 1

Enter an element to be pushed: 22

~~~~~Menu~~~~~ :

- => 1. Push an Element to Stack and Overflow demo
- => 2. Pop an Element from Stack and Underflow demo
- => 3. Palindrome demo
- => 4. Display
- => 5. Exit

Enter your choice: 2

Element popped is: 22

~~~~~Menu~~~~~ :

- => 1. Push an Element to Stack and Overflow demo
- => 2. Pop an Element from Stack and Underflow demo
- => 3. Palindrome demo

=> 4. Display

=> 5. Exit

Enter your choice: 3

Stack content are:

| 11 |

Reverse of stack content are:

| 11 |

It is a palindrome sequence

~~~~~Menu~~~~~ :

=> 1. Push an Element to Stack and Overflow demo

=> 2. Pop an Element from Stack and Underflow demo

=> 3. Palindrome demo

=> 4. Display

=> 5. Exit

Enter your choice: 4

Stack elements are:

| 11 |

~~~~~Menu~~~~~ :

=> 1. Push an Element to Stack and Overflow demo

=> 2. Pop an Element from Stack and Underflow demo

=> 3. Palindrome demo

=> 4. Display

=> 5. Exit

Enter your choice: 5

=== Code Execution Successful ===

Exp. No. 3

Develop a program in C for simulating a Printer Queue using Queue data structure with the following operations:

- a. Add a print job to the Printer Queue.**
- b. Process (delete) a print job from the Printer Queue.**
- c. Display all pending print jobs in the Queue.**
- d. Display the number of print jobs currently waiting in the Queue.**
- e. Demonstrate Queue overflow and underflow conditions.**
- f. Exit.**

CODE:

```
#include <stdio.h>

#include <stdlib.h>

#define MAX 5 // Maximum capacity of the printer queue

// Global Queue variables

int printerQueue[MAX];

int front = -1;

int rear = -1;

// Operation a & e: Add a print job (Enqueue) + Overflow demo

void addJob(int jobID) {

    if (rear == MAX - 1) {

        printf("\n>>> QUEUE OVERFLOW! The Printer Queue is FULL. Cannot add Job ID %d. <<<\n",
jobID);

        return;

    }

    if (front == -1) {

        front = 0; // Initialize front for the first insertion

    }

    rear++;

    printerQueue[rear] = jobID;

    printf("Successfully added Print Job ID: %d to the queue.\n", jobID);
```

```
}

// Operation b & e: Process a print job (Dequeue) + Underflow demo
void processJob() {
    if (front == -1 || front > rear) {
        printf("\n>>> QUEUE UNDERFLOW! The Printer Queue is EMPTY. No jobs to process. <<<\n");
        return;
    }
    printf("Processing and printing Job ID: %d ... Done!\n", printerQueue[front]);
    front++;
    // Reset queue pointers if all jobs are processed
    if (front > rear) {
        front = -1;
        rear = -1;
    }
}

// Operation c: Display all pending print jobs
void displayQueue() {
    if (front == -1 || front > rear) {
        printf("\nPrinter Queue is currently EMPTY!\n");
        return;
    }
    printf("\n=== PENDING PRINT JOBS IN QUEUE ===\n");
    printf("[Front] ");
    for (int i = front; i <= rear; i++) {
        printf("Job-%d ", printerQueue[i]);
    }
    printf("[Rear]\n");
}

// Operation d: Display number of print jobs currently waiting
void countJobs() {
```

```
if (front == -1 || front > rear) {  
    printf("\nTotal print jobs waiting in queue: 0\n");  
} else {  
    int count = (rear - front) + 1;  
    printf("\nTotal print jobs waiting in queue: %d\n", count);  
}  
}  
  
// Main Function with Menu Interface  
  
int main() {  
    int choice, jobID;  
    while (1) {  
        printf("\n===== \n");  
        printf("    PRINTER QUEUE SIMULATION MENU    \n");  
        printf("===== \n");  
        printf("1. Add a Print Job (Enqueue)\n");  
        printf("2. Process a Print Job (Dequeue)\n");  
        printf("3. Display All Pending Print Jobs\n");  
        printf("4. Display Count of Waiting Print Jobs\n");  
        printf("5. Exit\n");  
        printf("----- \n");  
        printf("Enter your choice (1-5): ");  
        scanf("%d", &choice);  
        switch (choice) {  
            case 1:  
                printf("Enter Print Job ID (Integer): ");  
                scanf("%d", &jobID);  
                addJob(jobID);  
                break;  
            case 2: processJob();  
                break;  

```

```
        case 3: displayQueue();
            break;

        case 4:
            countJobs();
            break;

        case 5:
            printf("\nExiting Printer Queue Simulator. Goodbye!\n");
            exit(0);

        default:
            printf("\nInvalid choice! Please enter a option between 1 and 5.\n");
    }
}

return 0;
}
```

OUTPUT:

```
=====
PRINTER QUEUE SIMULATION MENU
=====

1. Add a Print Job (Enqueue)
2. Process a Print Job (Dequeue)
3. Display All Pending Print Jobs
4. Display Count of Waiting Print Jobs
5. Exit
-----

Enter your choice (1-5): 1
Enter Print Job ID (Integer): 25
```

Successfully added Print Job ID: 25 to the queue.

=====

PRINTER QUEUE SIMULATION MENU

=====

1. Add a Print Job (Enqueue)
2. Process a Print Job (Dequeue)
3. Display All Pending Print Jobs
4. Display Count of Waiting Print Jobs
5. Exit

Enter your choice (1-5): 2

Processing and printing Job ID: 25 ... Done!

=====

PRINTER QUEUE SIMULATION MENU

=====

1. Add a Print Job (Enqueue)
2. Process a Print Job (Dequeue)
3. Display All Pending Print Jobs
4. Display Count of Waiting Print Jobs
5. Exit

Enter your choice (1-5): 3

Printer Queue is currently EMPTY!

=====

PRINTER QUEUE SIMULATION MENU

=====

1. Add a Print Job (Enqueue)
2. Process a Print Job (Dequeue)
3. Display All Pending Print Jobs

4. Display Count of Waiting Print Jobs

5. Exit

Enter your choice (1-5): 4

Total print jobs waiting in queue: 0

=====

PRINTER QUEUE SIMULATION MENU

=====

1. Add a Print Job (Enqueue)

2. Process a Print Job (Dequeue)

3. Display All Pending Print Jobs

4. Display Count of Waiting Print Jobs

5. Exit

Enter your choice (1-5): 5

Exiting Printer Queue Simulator. Goodbye!

=== Code Execution Successful ===

Exp. No. 4

Develop a menu driven Program in C for the following operations on Doubly Linked List (DLL) of Employee Data with the fields: SSN, Name, Dept, Designation, Sal, PhNo etc.

- a. Create a DLL of N Employees Data by using end insertion.**
- b. Display the status of DLL and count the number of nodes in it**
- c. Perform Insertion and Deletion at End of DLL**
- d. Perform Insertion and Deletion at Front of DLL**
- e. Demonstrate how this DLL can be used as Double Ended Queue.**
- f. Exit**

CODE:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

// Structure for Student Record Node

struct Student {

    char usn[20];

    char name[50];

    char program[20];

    int sem;

    char phno[15];

    struct Student *next;

};

// Global Head Pointer

struct Student *head = NULL;

// Safe string reader function to replace gets()

void readString(char *str, int size) {

    fgets(str, size, stdin);
```

```
// Remove trailing newline character added by fgets

size_t len = strlen(str);

if (len > 0 && str[len - 1] == '\n') {
    str[len - 1] = '\0';
}
}

// Helper function to create a new student node with user input
struct Student* createNode() {
    struct Student *newNode = (struct Student*) malloc(sizeof(struct Student));
    if (newNode == NULL) {
        printf("\nMemory allocation failed!\n");
        return NULL;
    }
    printf("\nEnter USN: ");
    scanf("%s", newNode->usn);
    printf("Enter Name: ");
    getchar(); // Clear leftover newline character from buffer
    readString(newNode->name, sizeof(newNode->name));
    printf("Enter Program (e.g., CSE): ");
    scanf("%s", newNode->program);
    printf("Enter Semester: ");
    scanf("%d", &newNode->sem);
    printf("Enter Phone Number: ");
    scanf("%s", newNode->phno);
    newNode->next = NULL;
    return newNode;
}
```

// Operation a & c: Insert node at the Front / Beginning

```
void insertFront() {  
    struct Student *newNode = createNode();  
    if (newNode == NULL) return;  
    newNode->next = head;  
    head = newNode;  
    printf("\n>>> Student record added successfully at the FRONT! <<<\n");  
}
```

// Operation c: Insert node at the End

```
void insertEnd() {  
    struct Student *newNode = createNode();  
    if (newNode == NULL) return;  
    if (head == NULL) {  
        head = newNode;  
    } else {  
        struct Student *temp = head;  
        while (temp->next != NULL) {  
            temp = temp->next;  
        }  
        temp->next = newNode;  
    }  
    printf("\n>>> Student record added successfully at the END! <<<\n");  
}
```

// Operation c: Delete node from the Front / Beginning

```
void deleteFront() {  
    if (head == NULL) {  
        printf("\n>>> List is EMPTY! No records to delete. <<<\n");  
    }
```

```
return;

}

struct Student *temp = head;

head = head->next;

printf("\n>>> Deleted Student Record - USN: %s, Name: %s <<<\n", temp->usn, temp->name);

free(temp);

}

// Operation c: Delete node from the End

void deleteEnd() {

    if (head == NULL) {

        printf("\n>>> List is EMPTY! No records to delete. <<<\n");

        return;

    }

    if (head->next == NULL) {

        printf("\n>>> Deleted Student Record - USN: %s, Name: %s <<<\n", head->usn, head->name);

        free(head);

        head = NULL;

        return;

    }

    struct Student *temp = head;

    struct Student *prev = NULL;

    while (temp->next != NULL) {

        prev = temp;

        temp = temp->next;

    }

    prev->next = NULL;

    printf("\n>>> Deleted Student Record - USN: %s, Name: %s <<<\n", temp->usn, temp->name);
```

```
free(temp);

}

// Operation b: Display all records and count total nodes

void display() {

    if (head == NULL) {

        printf("\n>>> Student Records List is EMPTY! <<<\n");

        return;

    }

    struct Student *temp = head;

    int count = 0;

    printf("\n===== STUDENT RECORDS LIST\n");

    printf("%-5s %-15s %-25s %-12s %-8s %-15s\n", "Sl.No", "USN", "Name", "Program", "Sem",
"Phone No");

    printf("-----\n")

    while (temp != NULL) {

        count++;

        printf("%-5d %-15s %-25s %-12s %-8d %-15s\n",

            count, temp->usn, temp->name, temp->program, temp->sem, temp->phno);

        temp = temp->next;

    }

    printf("=====\n");

    printf("Total Student Records (Node Count): %d\n", count);

}

// Operation d: Search for a student record by USN

void search() {

    if (head == NULL) {
```

```
printf("\n>>> List is EMPTY! Cannot perform search. <<<\n");

    return;

}

char searchUSN[20];

int found = 0, position = 1;

printf("\nEnter USN to search: ");

scanf("%s", searchUSN);

struct Student *temp = head

while (temp != NULL) {

    if (strcmp(temp->usn, searchUSN) == 0) {

        printf("\n>>> STUDENT RECORD FOUND (at position %d) <<<\n", position);

        printf("USN    : %s\n", temp->usn);

        printf("Name    : %s\n", temp->name);

        printf("Program : %s\n", temp->program);

        printf("Semester : %d\n", temp->sem);

        printf("Phone No : %s\n", temp->phno);

        found = 1;

        break;

    }

    temp = temp->next;

    position++;

}

if (!found) {

    printf("\n>>> Student record with USN '%s' NOT FOUND in the list! <<<\n", searchUSN);

}

}
```

// Main Function with Menu Interface

```
int main() {

    int choice;

    while (1) {

        printf("\n=====\\n");

        printf("  STUDENT SINGLY LINKED LIST MENU  \\n");

        printf("=====\\n");

        printf("1. Insert Student at Beginning (Front Insertion)\\n");

        printf("2. Insert Student at End\\n");

        printf("3. Delete Student from Beginning\\n");

        printf("4. Delete Student from End\\n");

        printf("5. Display Records & Count Nodes\\n");

        printf("6. Search Student Record by USN\\n");

        printf("7. Exit\\n");

        printf("-----\\n");

        printf("Enter your choice (1-7): ");

        scanf("%d", &choice);

        switch (choice) {

            case 1:

                insertFront();

                break;

            case 2:

                insertEnd();

                break;

            case 3:

                deleteFront();

                break;

            case 4:
```

```
        deleteEnd();

        break;

    case 5:

        display();

        break;

    case 6:

        search();

        break;

    case 7:

        printf("\nExiting Program. Freeing memory... Goodbye!\n");

        while (head != NULL) {

            struct Student *temp = head;

            head = head->next;

            free(temp);

        }

        exit(0);

    default:

        printf("\nInvalid Choice! Please enter a number between 1 and 7.\n");

    }

}

return 0;

}
```

OUTPUT:

```
=====

STUDENT SINGLY LINKED LIST MENU

=====
```

1. Insert Student at Beginning (Front Insertion)
2. Insert Student at End
3. Delete Student from Beginning
4. Delete Student from End
5. Display Records & Count Nodes
6. Search Student Record by USN
7. Exit

Enter your choice (1-7): 1

Enter USN: 1VJ26CS001

Enter Name: RAJINI M

Enter Program (e.g., CSE): CSE

Enter Semester: 3

Enter Phone Number: 676895045

>>> Student record added successfully at the FRONT! <<<

=====
STUDENT SINGLY LINKED LIST MENU

- =====
1. Insert Student at Beginning (Front Insertion)
2. Insert Student at End
3. Delete Student from Beginning
4. Delete Student from End
5. Display Records & Count Nodes
6. Search Student Record by USN
7. Exit

Enter your choice (1-7): 2

Enter USN: 1VJ26AI004

Enter Name: ANAJALI D K

Enter Program (e.g., CSE): AI&ML

Enter Semester: 5

Enter Phone Number: 9808086543

>>> Student record added successfully at the END! <<

=====

STUDENT SINGLY LINKED LIST MENU

=====

1. Insert Student at Beginning (Front Insertion)
2. Insert Student at End
3. Delete Student from Beginning
4. Delete Student from End
5. Display Records & Count Nodes
6. Search Student Record by USN
7. Exit

Enter your choice (1-7): 3

>>> Deleted Student Record - USN: 1VJ26CS001, Name: RAJINI M <<<

=====

STUDENT SINGLY LINKED LIST MENU

=====

1. Insert Student at Beginning (Front Insertion)
2. Insert Student at End
3. Delete Student from Beginning
4. Delete Student from End
5. Display Records & Count Nodes

6. Search Student Record by USN

7. Exit

Enter your choice (1-7): 4

>>> Deleted Student Record - USN: 1VJ26AI004, Name: ANAJALI D K <<<

=====

STUDENT SINGLY LINKED LIST MENU

=====

1. Insert Student at Beginning (Front Insertion)

2. Insert Student at End

3. Delete Student from Beginning

4. Delete Student from End

5. Display Records & Count Nodes

6. Search Student Record by USN

7. Exit

Enter your choice (1-7): 5

>>> Student Records List is EMPTY! <<<

=====

STUDENT SINGLY LINKED LIST MENU

=====

1. Insert Student at Beginning (Front Insertion)

2. Insert Student at End

3. Delete Student from Beginning

4. Delete Student from End

5. Display Records & Count Nodes

6. Search Student Record by USN

7. Exit

Enter your choice (1-7): 6

>>>List is EMPTY! Cannot perform search. <<<

=====
STUDENT SINGLY LINKED LIST MENU

=====
1. Insert Student at Beginning (Front Insertion)

2. Insert Student at End

3. Delete Student from Beginning

4. Delete Student from End

5. Display Records & Count Nodes

6. Search Student Record by USN

7. Exit

Enter your choice (1-7): 7

Exiting Program. Freeing memory... Goodbye!

=== Code Execution Successful ===

Exp. No. 5

Develop a Program in C for the following:

a. Create a Binary Tree by inserting integer elements entered by the user in level order.

b. Implement the following functions:

i. createTree() – Create a binary tree using dynamic memory allocation.

ii. preorder() – Display the elements using Preorder Traversal.

iii. inorder() – Display the elements using Inorder Traversal.

iv. postorder() – Display the elements using Postorder Traversal.

c. Provide a menu-driven interface with the following options:

- i. Create Binary Tree**
- ii. Display Preorder Traversal**
- iii. Display Inorder Traversal**
- iv. Display Postorder Traversal**
- v. Exit**

CODE:

```
#include <stdio.h>

#include <stdlib.h>

// Structure for a Binary Tree Node

struct Node {

    int data;

    struct Node *left;

    struct Node *right;

};

// Queue node structure for Level Order insertion

struct QueueNode {

    struct Node *treeNode;

    struct QueueNode *next;

};

// Queue front and rear pointers

struct QueueNode *front = NULL;

struct QueueNode *rear = NULL;

// Helper function to create a new Binary Tree Node

struct Node* createNode(int val) {

    struct Node *newNode = (struct Node*) malloc(sizeof(struct Node));

    newNode->data = val;
```

```
newNode->left = NULL;

newNode->right = NULL;

return newNode;

}

// Queue Enqueue function

void enqueue(struct Node *tNode) {

    struct QueueNode *temp = (struct QueueNode*) malloc(sizeof(struct QueueNode));

    temp->treeNode = tNode;

    temp->next = NULL;

    if (rear == NULL) {

        front = rear = temp;

        return;

    }

    rear->next = temp;

    rear = temp;

}

// Queue Dequeue function

struct Node* dequeue() {

    if (front == NULL) return NULL;

    struct QueueNode *temp = front;

    struct Node *tNode = temp->treeNode

    front = front->next;

    if (front == NULL) rear = NULL

    free(temp);

    return tNode;

}

// Helper function to clear remaining queue elements
```

```
void clearQueue() {  
    while (front != NULL) {  
        dequeue();  
    }  
}  
  
// Function i: createTree() - Create a binary tree in Level Order  
struct Node* createTree(struct Node *root) {  
    int n, val;  
  
    printf("\nEnter total number of elements to insert in Binary Tree: ");  
  
    scanf("%d", &n);  
  
    if (n <= 0) {  
        printf("\nInvalid number of elements!\n");  
        return root;  
    }  
  
    clearQueue(); // Clear queue before starting  
  
    for (int i = 0; i < n; i++) {  
        printf("Enter element %d: ", i + 1);  
  
        scanf("%d", &val);  
  
        struct Node *newNode = createNode(val);  
  
        if (root == NULL) {  
            root = newNode; // First element becomes the root  
        } else {  
            // Get the parent node from front of the queue  
  
            struct Node *parent = front->treeNode;  
  
            if (parent->left == NULL) {  
                parent->left = newNode;  
            } else if (parent->right == NULL) {
```

```
parent->right = newNode;

    dequeue(); // Parent now has both left and right children, so remove from queue

}

}

enqueue(newNode); // Add new node to queue for future child attachments

}

printf("\n>>> Binary Tree created successfully in Level Order! <<<\n");

return root;

}

// Function ii: preorder() - Display elements in Preorder Traversal (Root -> Left -> Right)

void preorder(struct Node *root) {

    if (root != NULL) {

        printf("%d ", root->data);

        preorder(root->left);

        preorder(root->right);

    }

}

// Function iii: inorder() - Display elements in Inorder Traversal (Left -> Root -> Right)

void inorder(struct Node *root) {

    if (root != NULL) {

        inorder(root->left);

        printf("%d ", root->data);

        inorder(root->right);

    }

}

// Function iv: postorder() - Display elements in Postorder Traversal (Left -> Right -> Root)

void postorder(struct Node *root) {
```

```
if (root != NULL) {  
    postorder(root->left);  
    postorder(root->right);  
    printf("%d ", root->data);  
}  
}  
  
// Main Function with Menu Interface  
  
int main() {  
    struct Node *root = NULL;  
  
    int choice;  
  
    while (1) {  
        printf("\n=====\\n");  
        printf("    BINARY TREE OPERATIONS MENU    \\n");  
        printf("=====\\n");  
        printf("1. Create Binary Tree (Level Order)\\n");  
        printf("2. Display Preorder Traversal\\n");  
        printf("3. Display Inorder Traversal\\n");  
        printf("4. Display Postorder Traversal\\n");  
        printf("5. Exit\\n");  
        printf("-----\\n");  
        printf("Enter your choice (1-5): ");  
        scanf("%d", &choice);  
  
        switch (choice) {  
            case 1:  
                root = createTree(root);  
                break;  
            case 2:
```

```
if (root == NULL) {

    printf("\nTree is empty! Please create a tree first.\n");

} else {

    printf("\nPreorder Traversal: ");

    preorder(root);

    printf("\n");

}

break;

case 3:

    if (root == NULL) {

        printf("\nTree is empty! Please create a tree first.\n");

    } else {

        printf("\nInorder Traversal: ");

        inorder(root);

        printf("\n");

    }

    break;

case 4:

    if (root == NULL) {

        printf("\nTree is empty! Please create a tree first.\n");

    } else {

        printf("\nPostorder Traversal: ");

        postorder(root);

        printf("\n");

    }

    break;

case 5:
```

```
printf("\nExiting Program. Goodbye!\n");  
    exit(0)  
default:  
    printf("\nInvalid Choice! Please enter a number between 1 and 5.\n");  
}  
}  
return 0;  
}
```

OUTPUT:

=====

BINARY TREE OPERATIONS MENU

=====

1. Create Binary Tree (Level Order)
2. Display Preorder Traversal
3. Display Inorder Traversal
4. Display Postorder Traversal
5. Exit

Enter your choice (1-5): 1

Enter total number of elements to insert in Binary Tree: 4

Enter element 1: 8

Enter element 2: 6

Enter element 3: 5

Enter element 4: 7

>>> Binary Tree created successfully in Level Order! <<<

=====

BINARY TREE OPERATIONS MENU

=====

1. Create Binary Tree (Level Order)

2. Display Preorder Traversal
3. Display Inorder Traversal
4. Display Postorder Traversal
5. Exit

Enter your choice (1-5): 2

Preorder Traversal: 8 6 7 5

=====

BINARY TREE OPERATIONS MENU

=====

1. Create Binary Tree (Level Order)
2. Display Preorder Traversal
3. Display Inorder Traversal
4. Display Postorder Traversal
5. Exit

Enter your choice (1-5): 3

Inorder Traversal: 7 6 8 5

=====

BINARY TREE OPERATIONS MENU

=====

1. Create Binary Tree (Level Order)
2. Display Preorder Traversal
3. Display Inorder Traversal
4. Display Postorder Traversal
5. Exit

Enter your choice (1-5): 4

Postorder Traversal: 7 6 5 8

=====

BINARY TREE OPERATIONS MENU

- =====
1. Create Binary Tree (Level Order)
 2. Display Preorder Traversal
 3. Display Inorder Traversal
 4. Display Postorder Traversal
 5. Exit
-

Enter your choice (1-5): 5

Exiting Program. Goodbye!

=== Code Execution Successful ===

Exp. No. 6

Develop a Program in C for the following operations on Graph(G) of Cities

a. Create an Adjacency Matrix representing the graph of N cities

b. Print all the nodes reachable from a given starting node in a digraph using DFS/BFS method.

CODE:

```
#include <stdio.h>

#include <stdlib.h>

#define MAX 20 // Maximum number of cities

// Global variables for Adjacency Matrix and Visited tracking
int adjMatrix[MAX][MAX];
int visited[MAX];
int numCities;

// Function b: DFS Traversal to find all reachable cities
void dfs(int currentCity) {
    int i;

    visited[currentCity] = 1; // Mark the current city as visited
```

```
for (i = 1; i <= numCities; i++) {

    // If there is a direct route/edge to city 'i' and it has not been visited yet
    if (adjMatrix[currentCity][i] == 1 && visited[i] == 0) {

        printf("-> City %d", i);

        dfs(i); // Recursive call for the next connected city
    }

}

}

// Function a: Create an Adjacency Matrix representing N cities
void createGraph() {

    int i, j;

    printf("\nEnter the number of cities (N): ");

    scanf("%d", &numCities);

    if (numCities <= 0 || numCities > MAX) {

        printf("Invalid number of cities! Please enter between 1 and %d.\n", MAX);

        return;

    }

    printf("\nEnter the Adjacency Matrix (%dx%d):\n", numCities, numCities);

    printf("(Enter 1 if direct route exists between City i and City j, otherwise 0)\n\n");

    for (i = 1; i <= numCities; i++) {

        for (j = 1; j <= numCities; j++) {

            printf("Is there a route from City %d to City %d? (1/0): ", i, j);

            scanf("%d", &adjMatrix[i][j]);

        }

    }

    printf("\n>>> Graph of Cities created successfully! <<<\n");

}
```

// Function to display the Adjacency Matrix

```
void displayMatrix() {  
    int i, j;  
    if (numCities == 0) {  
        printf("\nGraph is empty! Please create the graph first.\n");  
        return;  
    }  
    printf("\n--- ADJACENCY MATRIX OF CITIES ---\n ");  
    for (i = 1; i <= numCities; i++) {  
        printf(" C%d", i);  
    }  
    printf("\n");  
    for (i = 1; i <= numCities; i++) {  
        printf("C%d ", i);  
        for (j = 1; j <= numCities; j++) {  
            printf(" %d ", adjMatrix[i][j]);  
        }  
        printf("\n");  
    }  
}
```

// Function to check reachability from a given starting city

```
void checkReachability() {  
    int startCity, i, reachableCount = 0;  
    if (numCities == 0) {  
        printf("\nGraph is empty! Please create the graph first.\n");  
        return;  
    }  
}
```

```
printf("\nEnter the starting City number (1 to %d): ", numCities);

scanf("%d", &startCity);

if (startCity < 1 || startCity > numCities) {
    printf("Invalid starting city number!\n");
    return;
}

// Initialize visited array to 0 (Unvisited)
for (i = 1; i <= numCities; i++) {
    visited[i] = 0;
}

printf("\nReachable Cities starting from City %d using DFS:\n", startCity);
printf("City %d", startCity);
dfs(startCity); // Call DFS traversal
printf("\n\n--- Summary of Reachable Cities ---\n");
for (i = 1; i <= numCities; i++) {
    if (visited[i] == 1) {
        printf("City %d is REACHABLE\n", i);
        reachableCount++;
    } else {
        printf("City %d is NOT Reachable\n", i);
    }
}

printf("Total cities reachable from City %d = %d\n", startCity, reachableCount);

// Main Function with Menu Interface
int main() {
    int choice;
```

```
while (1) {

    printf("\n=====\\n");

    printf("    GRAPH OF CITIES OPERATIONS    \\n");

    printf("=====\\n");

    printf("1. Create Adjacency Matrix for N Cities\\n");

    printf("2. Display Adjacency Matrix\\n");

    printf("3. Find Reachable Cities (DFS Method)\\n");

    printf("4. Exit\\n");

    printf("-----\\n");

    printf("Enter your choice (1-4): ");

    scanf("%d", &choice);

switch (choice) {

    case 1:

        createGraph();

        break;

    case 2:

        displayMatrix();

        break;

    case 3:

        checkReachability();

        break;

    case 4:

        printf("\\nExiting Program. Goodbye!\\n");

        exit(0);

    default:

        printf("\\nInvalid Choice! Please select between 1 and 4.\\n");

}

}
```

```
}  
  
return 0;  
  
}
```

OUTPUT:

=====

GRAPH OF CITIES OPERATIONS

=====

1. Create Adjacency Matrix for N Cities
2. Display Adjacency Matrix
3. Find Reachable Cities (DFS Method)
4. Exit

Enter your choice (1-4): 1

Enter the number of cities (N): 2

Enter the Adjacency Matrix (2x2):

(Enter 1 if direct route exists between City i and City j, otherwise 0)

Is there a route from City 1 to City 1? (1/0): 0

Is there a route from City 1 to City 2? (1/0): 1

Is there a route from City 2 to City 1? (1/0): 1

Is there a route from City 2 to City 2? (1/0): 0

>>> Graph of Cities created successfully! <<<

=====

GRAPH OF CITIES OPERATIONS

=====

1. Create Adjacency Matrix for N Cities
2. Display Adjacency Matrix
3. Find Reachable Cities (DFS Method)

4. Exit

Enter your choice (1-4): 2

--- ADJACENCY MATRIX OF CITIES ---

C1 C2

C1 0 1

C2 1 0

=====
GRAPH OF CITIES OPERATIONS

=====
1. Create Adjacency Matrix for N Cities

2. Display Adjacency Matrix

3. Find Reachable Cities (DFS Method)

4. Exit

Enter your choice (1-4): 3

Enter the starting City number (1 to 2): 1

Reachable Cities starting from City 1 using DFS:

City 1 -> City 2

--- Summary of Reachable Cities ---

City 1 is REACHABLE

City 2 is REACHABLE

Total cities reachable from City 1 = 2

=====
GRAPH OF CITIES OPERATIONS

=====
1. Create Adjacency Matrix for N Cities

2. Display Adjacency Matrix
3. Find Reachable Cities (DFS Method)
4. Exit

Enter your choice (1-4): 4

Exiting Program. Goodbye!

=== Code Execution Successful ===

Exp. No. 1**Develop a C program to:**

- a. Read two sparse matrices of the same order.**
- b. Convert both into 3-tuple representation.**
- c. Perform matrix addition.**
- d. Display the resultant matrix in both normal matrix form and 3-tuple form.**

CODE:

```
#include <stdio.h>

#include <stdlib.h>

#define MAX 100 // Maximum non-zero elements allowed

// Structure to represent 3-tuple element
struct Sparse {
    int row;
    int col;
    int val;
};

// Function to convert a standard 2D matrix into a 3-tuple representation
// Returns the total number of non-zero elements

int createTuple(int mat[20][20], int rows, int cols, struct Sparse tuple[]) {
    int k = 1; // tuple[0] stores overall dimensions & total non-zero count
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            if (mat[i][j] != 0) {
                tuple[k].row = i;
                tuple[k].col = j;
                tuple[k].val = mat[i][j];
                k++;
            }
        }
    }
}
```

```
    }

}

// Set metadata in the first row (index 0)

tuple[0].row = rows;

tuple[0].col = cols;

tuple[0].val = k - 1; // Total non-zero elements count

return k - 1;

}

// Function to display a 3-tuple representation table

void displayTuple(struct Sparse tuple[], char name[]) {

    printf("\n--- 3-Tuple Representation of %s ---\n", name);

    printf("%-8s %-8s %-8s\n", "Row", "Column", "Value");

    printf("-----\n");

    // First row shows total rows, total columns, and total non-zero elements

    for (int i = 0; i <= tuple[0].val; i++) {

        printf("%-8d %-8d %-8d\n", tuple[i].row, tuple[i].col, tuple[i].val);

    }

}

// Function to add two sparse matrices using 3-tuple representation

void addSparse(struct Sparse t1[], struct Sparse t2[], struct Sparse result[]) {

    int i = 1, j = 1, k = 1;

    // Set metadata for resultant matrix

    result[0].row = t1[0].row;

    result[0].col = t1[0].col;

    while (i <= t1[0].val && j <= t2[0].val) {

        // Compare row positions first

        if (t1[i].row < t2[j].row) {
```

```
result[k++] = t1[i++];

    } else if (t1[i].row > t2[j].row) {

        result[k++] = t2[j++];

    } else { // Rows are equal, compare column positions

        if (t1[i].col < t2[j].col) {

            result[k++] = t1[i++];

        } else if (t1[i].col > t2[j].col) {

            result[k++] = t2[j++];

        } else { // Both row and column match: Add values

            int sum = t1[i].val + t2[j].val;

            if (sum != 0) { // Store only non-zero sum

                result[k].row = t1[i].row;

                result[k].col = t1[i].col;

                result[k].val = sum;

                k++;

            }

            i++;

            j++;

        }

    }

}

// Copy remaining elements of t1, if any
while (i <= t1[0].val) {

    result[k++] = t1[i++];

}

// Copy remaining elements of t2, if any
while (j <= t2[0].val) {
```

```
result[k++] = t2[j++];

}

// Store total count of non-zero elements in result metadata

result[0].val = k - 1;

}

// Function to reconstruct and display the resultant matrix in normal 2D matrix format

void displayNormalMatrix(struct Sparse result[]) {

    int rows = result[0].row;

    int cols = result[0].col;

    int matrix[20][20] = {0}; // Initialize all elements to 0

    // Fill non-zero values from 3-tuple result

    for (int i = 1; i <= result[0].val; i++) {

        matrix[result[i].row][result[i].col] = result[i].val;

    }

    printf("\n--- Resultant Sparse Matrix (Normal 2D Form) ---\n");

    for (int i = 0; i < rows; i++) {

        for (int j = 0; j < cols; j++) {

            printf("%4d ", matrix[i][j]);

        }

        printf("\n");

    }

}

int main() {

    int rows, cols;

    int mat1[20][20], mat2[20][20];

    struct Sparse tuple1[MAX], tuple2[MAX], resultTuple[MAX];

    printf("=====\n");
```

```
printf("    SPARSE MATRIX ADDITION (3-TUPLE)  \n");

printf("=====\\n");

printf("Enter number of rows and columns for the matrices: ");

scanf("%d %d", &rows, &cols);

// Read Matrix 1

printf("\\nEnter elements of Matrix 1 (%dx%d):\\n", rows, cols);

for (int i = 0; i < rows; i++) {

    for (int j = 0; j < cols; j++) {

        scanf("%d", &mat1[i][j]);

    }

}

// Read Matrix 2

printf("\\nEnter elements of Matrix 2 (%dx%d):\\n", rows, cols);

for (int i = 0; i < rows; i++) {

    for (int j = 0; j < cols; j++) {

        scanf("%d", &mat2[i][j]);

    }

}

// Step b: Convert to 3-tuple form

createTuple(mat1, rows, cols, tuple1);

createTuple(mat2, rows, cols, tuple2);

// Display input 3-tuples

displayTuple(tuple1, "Matrix 1");

displayTuple(tuple2, "Matrix 2");

// Step c: Perform addition

addSparse(tuple1, tuple2, resultTuple);

// Step d: Display result in 3-tuple and normal forms
```

```
displayTuple(resultTuple, "Resultant Matrix (Sum)");

displayNormalMatrix(resultTuple);

return 0;

}
```

OUTPUT:

=====

SPARSE MATRIX ADDITION (3-TUPLE)

=====

Enter number of rows and columns for the matrices: 3

3

Enter elements of Matrix 1 (3x3):

0 0 5

0 3 0

1 0 0

Enter elements of Matrix 2 (3x3):

0 0 2

4 0 0

1 0 7

--- 3-Tuple Representation of Matrix 1 ---

| Row | Column | Value |
|-----|--------|-------|
|-----|--------|-------|

| | | |
|---|---|---|
| 3 | 3 | 3 |
|---|---|---|

| | | |
|---|---|---|
| 0 | 2 | 5 |
|---|---|---|

| | | |
|---|---|---|
| 1 | 1 | 3 |
|---|---|---|

| | | |
|---|---|---|
| 2 | 0 | 1 |
|---|---|---|

--- 3-Tuple Representation of Matrix 2 ---

| Row | Column | Value |
|-----|--------|-------|
|-----|--------|-------|

| | | |
|---|---|---|
| 3 | 3 | 4 |
|---|---|---|

| | | |
|---|---|---|
| 0 | 2 | 2 |
|---|---|---|

| | | |
|---|---|---|
| 1 | 0 | 4 |
|---|---|---|

| | | |
|---|---|---|
| 2 | 0 | 1 |
|---|---|---|

| | | |
|---|---|---|
| 2 | 2 | 7 |
|---|---|---|

--- 3-Tuple Representation of Resultant Matrix (Sum) ---

| Row | Column | Value |
|-----|--------|-------|
|-----|--------|-------|

| | | |
|---|---|---|
| 3 | 3 | 5 |
|---|---|---|

| | | |
|---|---|---|
| 0 | 2 | 7 |
|---|---|---|

| | | |
|---|---|---|
| 1 | 0 | 4 |
|---|---|---|

| | | |
|---|---|---|
| 1 | 1 | 3 |
|---|---|---|

| | | |
|---|---|---|
| 2 | 0 | 2 |
|---|---|---|

| | | |
|---|---|---|
| 2 | 2 | 7 |
|---|---|---|

--- Resultant Sparse Matrix (Normal 2D Form) ---

| | | |
|---|---|---|
| 0 | 0 | 7 |
|---|---|---|

| | | |
|---|---|---|
| 4 | 3 | 0 |
|---|---|---|

| | | |
|---|---|---|
| 2 | 0 | 7 |
|---|---|---|

=== Code Execution Successful ===

Exp. No. 2

Develop a C-based Expression Conversion tool that uses stack data structure to convert arithmetic expressions from infix notation to postfix notation. The tool should support alphanumeric operands, parentheses, and the operators (+, -, *, /, %, ^). Analyze the correctness of the conversion by testing various valid expressions.

CODE:

```
#include <stdio.h>

#include <stdlib.h>

#include <ctype.h>

#include <string.h>

#define MAX 100

// Stack definition

char stack[MAX];

int top = -1;

// Push function

void push(char item) {

    if (top >= MAX - 1) {

        printf("Stack Overflow!\n");

        return;

    }

    stack[++top] = item;

}

// Pop function

char pop() {

    if (top < 0) {

        return '\0'; // Return null character if stack is empty

    }

    return stack[top--];

}
```

```
}

// Peek function to check top element without popping

char peek() {
    if (top < 0) return '\0';
    return stack[top];
}

// Function to return precedence of operators

int precedence(char op) {
    switch (op) {
        case '+':
        case '-':
            return 1;
        case '*':
        case '/':
        case '%':
            return 2;
        case '^':
            return 3;
        default:
            return 0;
    }
}

// Function to check if operator is Right-Associative

int isRightAssociative(char op) {
    if (op == '^') return 1;
    return 0;
}
```

// Function to convert Infix expression to Postfix

```
void infixToPostfix(char infix[], char postfix[]) {  
  
    int i = 0, k = 0;  
  
    char symbol;  
  
    while ((symbol = infix[i++]) != '\0') {  
  
        // Ignore whitespace and newline characters  
  
        if (symbol == ' ' || symbol == '\t' || symbol == '\n') {  
  
            continue;  
  
        }  
  
        // 1. If symbol is an alphanumeric operand, add it directly to postfix output  
  
        if (isalnum(symbol)) {  
  
            postfix[k++] = symbol;  
  
        }  
  
        // 2. If symbol is '(', push it onto the stack  
  
        else if (symbol == '(') {  
  
            push(symbol);  
  
        }  
  
        // 3. If symbol is ')', pop from stack until '(' is found  
  
        else if (symbol == ')') {  
  
            while (top != -1 && peek() != '(') {  
  
                postfix[k++] = pop();  
  
            }  
  
            pop(); // Remove '(' from stack  
  
        }  
  
        // 4. If symbol is an operator  
  
        else if (symbol == '+' || symbol == '-' || symbol == '*' ||  
  
            symbol == '/' || symbol == '%' || symbol == '^') {
```

```
while (top != -1 && precedence(peek()) > 0) {

    char topOp = peek();

    int precTop = precedence(topOp);

    int precCurr = precedence(symbol);

    // For Left-Associative: pop if top precedence >= current precedence
    // For Right-Associative (^): pop if top precedence > current precedence

    if ((!isRightAssociative(symbol) && precTop >= precCurr) ||
        (isRightAssociative(symbol) && precTop > precCurr)) {

        postfix[k++] = pop();

    } else {

        break;

    }

}

push(symbol);

}

}

// Pop all remaining operators from stack

while (top != -1) {

    postfix[k++] = pop();

}

postfix[k] = '\0'; // Null-terminate postfix string

}

int main() {

    char infix[MAX], postfix[MAX];

    printf("=====\n");

    printf(" INFIX TO POSTFIX EXPRESSION CONVERSION TOOL  \n");

    printf("=====\n\n");
```

```

printf("Enter a valid Infix Expression: ");

fgets(infix, sizeof(infix), stdin);

infixToPostfix(infix, postfix);

printf("\nInfix Expression  : %s", infix);

printf("Postfix Expression : %s\n", postfix);

return 0;

}

```

OUTPUT:

```

=====

INFIX TO POSTFIX EXPRESSION CONVERSION TOOL

=====

Enter a valid Infix Expression: A+B*C-D/E

Infix Expression  : A+B*C-D/E

Postfix Expression : ABC*+DE/-

=== Code Execution Successful ===

```

Exp. No. 3

Develop a menu-driven Program in C for the following operations on Circular Queue (integer/floating point data):

- a. Insert an element into the Circular Queue.**
- b. Delete an element from the Circular Queue.**
- c. Display the contents of the Circular Queue.**
- d. Demonstrate overflow and underflow conditions.**
- e. Exit.**

CODE:

```

#include <stdio.h>

#include <stdlib.h>

#define MAX 5 // Maximum capacity of the Circular Queue

```

```
// Global Circular Queue array and pointers
```

```
double cq[MAX];
```

```
int front = -1;
```

```
int rear = -1;
```

```
// Operation a & d: Insert an element into Circular Queue + Overflow Demo
```

```
void insert(double val) {
```

```
    // Overflow Condition: when rear is just behind front circularly
```

```
    if ((front == 0 && rear == MAX - 1) || (front == rear + 1)) {
```

```
        printf("\n>>> CIRCULAR QUEUE OVERFLOW! Queue is FULL. Cannot insert %.2f  
<<<\n", val);
```

```
        return;
```

```
    }
```

```
    // First element insertion
```

```
    if (front == -1) {
```

```
        front = 0;
```

```
        rear = 0;
```

```
    } else {
```

```
        rear = (rear + 1) % MAX; // Circular increment
```

```
    }
```

```
    cq[rear] = val;
```

```
    printf("Successfully inserted %.2f into Circular Queue.\n", val);
```

```
}
```

```
// Operation b & d: Delete an element from Circular Queue + Underflow Demo
```

```
void deleteElement() {
```

```
    // Underflow Condition: queue is empty
```

```
    if (front == -1) {
```

```
printf("\n>>> CIRCULAR QUEUE UNDERFLOW! Queue is EMPTY. Nothing to delete.
<<<\n");
```

```
    return;
```

```
}
```

```
printf("Deleted element: %.2f\n", cq[front]);
```

```
// If only one element was present
```

```
if (front == rear) {
```

```
    front = -1;
```

```
    rear = -1;
```

```
} else {
```

```
    front = (front + 1) % MAX; // Circular increment
```

```
}
```

```
}
```

```
// Operation c: Display the contents of Circular Queue
```

```
void display() {
```

```
    if (front == -1) {
```

```
        printf("\nCircular Queue is currently EMPTY!\n");
```

```
        return;
```

```
}
```

```
printf("\n--- CIRCULAR QUEUE CONTENTS ---\n");
```

```
printf("[Front: Index %d] -> ", front);
```

```
int i = front;
```

```
while (1) {
```

```
    printf("[%d] ", cq[i]);
```

```
    if (i == rear) break;
```

```
    i = (i + 1) % MAX;
```

```
    }

    printf("<- [Rear: Index %d]\n", rear);
}

// Main Function with Menu Interface
int main() {
    int choice;
    double val;
    while (1) {
        printf("\n===== \n");
        printf("  CIRCULAR QUEUE OPERATIONS MENU  \n");
        printf("===== \n");
        printf("1. Insert an Element (Enqueue)\n");
        printf("2. Delete an Element (Dequeue)\n");
        printf("3. Display Contents of Circular Queue\n");
        printf("4. Exit\n");
        printf("----- \n");
        printf("Enter your choice (1-4): ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter value to insert (integer or decimal): ");
                scanf("%lf", &val);
                insert(val);
                break;
            case 2:
                deleteElement();
```

```
        break;

    case 3:

        display();

        break;

    case 4:

        printf("\nExiting Circular Queue Program. Goodbye!\n");

        exit(0);

    default:

        printf("\nInvalid choice! Please select between 1 and 4.\n");

    }

}

return 0;

}
```

OUTPUT:

=====

CIRCULAR QUEUE OPERATIONS MENU

=====

1. Insert an Element (Enqueue)
2. Delete an Element (Dequeue)
3. Display Contents of Circular Queue
4. Exit

Enter your choice (1-4): 1

Enter value to insert (integer or decimal): 1

Successfully inserted 1.00 into Circular Queue.

=====

CIRCULAR QUEUE OPERATIONS MENU

=====

1. Insert an Element (Enqueue)
2. Delete an Element (Dequeue)
3. Display Contents of Circular Queue
4. Exit

Enter your choice (1-4): 2

Deleted element: 1.00

=====

CIRCULAR QUEUE OPERATIONS MENU

=====

1. Insert an Element (Enqueue)
2. Delete an Element (Dequeue)
3. Display Contents of Circular Queue
4. Exit

Enter your choice (1-4): 1

Enter value to insert (integer or decimal): 5

Successfully inserted 5.00 into Circular Queue.

=====

CIRCULAR QUEUE OPERATIONS MENU

=====

1. Insert an Element (Enqueue)
2. Delete an Element (Dequeue)

3. Display Contents of Circular Queue

4. Exit

Enter your choice (1-4):

3

--- CIRCULAR QUEUE CONTENTS ---

[Front: Index 0] -> [5.00] <- [Rear: Index 0]

=====

CIRCULAR QUEUE OPERATIONS MENU

=====

1. Insert an Element (Enqueue)

2. Delete an Element (Dequeue)

3. Display Contents of Circular Queue

4. Exit

Enter your choice (1-4): 4

Exiting Circular Queue Program. Goodbye!

=== Code Execution Successful ===

Exp. No. 4

Develop a menu driven Program in C for the following operations on Doubly Linked List (DLL) of Employee Data with the fields: SSN, Name, Dept, Designation, Sal, PhNo etc.

- a. Create a DLL of N Employees Data by using end insertion.**
- b. Display the status of DLL and count the number of nodes in it**
- c. Perform Insertion and Deletion at End of DLL**
- d. Perform Insertion and Deletion at Front of DLL**
- e. Demonstrate how this DLL can be used as Double Ended Queue.**

f. Exit**CODE:**

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

// Structure for Employee Doubly Linked List Node

struct Employee {

    char ssn[15];

    char name[50];

    char dept[20];

    char desig[20];

    float sal;

    char phno[15];

    struct Employee *prev;

    struct Employee *next;

};

// Global Head Pointer

struct Employee *head = NULL;

// Helper function to safely read string input replacing gets()

void readString(char *str, int size) {

    fgets(str, size, stdin);

    size_t len = strlen(str);

    if (len > 0 && str[len - 1] == '\n') {

        str[len - 1] = '\0';

    }

}
```

// Helper function to create a new Employee node with user input

```
struct Employee* createNode() {  
    struct Employee *newNode = (struct Employee*) malloc(sizeof(struct Employee));  
    if (newNode == NULL) {  
        printf("\nMemory allocation failed!\n");  
        return NULL;  
    }  
    printf("\nEnter SSN: ");  
    scanf("%s", newNode->ssn);  
    printf("Enter Name: ");  
    getchar(); // Clear leftover newline  
    readString(newNode->name, sizeof(newNode->name));  
    printf("Enter Department: ");  
    scanf("%s", newNode->dept);  
    printf("Enter Designation: ");  
    scanf("%s", newNode->desig);  
    printf("Enter Salary: ");  
    scanf("%f", &newNode->sal);  
    printf("Enter Phone Number: ");  
    scanf("%s", newNode->phno);  
    newNode->prev = NULL;  
    newNode->next = NULL;  
    return newNode;  
}
```

// Operation d & e: Insert at Front

```
void insertFront() {
```

```
struct Employee *newNode = createNode();  
if (newNode == NULL) return;  
if (head == NULL) {  
    head = newNode;  
} else {  
    newNode->next = head;  
    head->prev = newNode;  
    head = newNode;  
}  
printf("\n>>> Employee record inserted successfully at FRONT! <<<\n");  
}
```

// Operation a, c & e: Insert at End

```
void insertEnd() {  
    struct Employee *newNode = createNode();  
    if (newNode == NULL) return;  
    if (head == NULL) {  
        head = newNode;  
    } else {  
        struct Employee *temp = head;  
        while (temp->next != NULL) {  
            temp = temp->next;  
        }  
        temp->next = newNode;  
        newNode->prev = temp;  
    }  
}
```

```
printf("\n>>> Employee record inserted successfully at END! <<<\n");
}

// Operation a: Create DLL of N Employees using End Insertion
void createDLL() {
    int n, i;

    printf("\nEnter number of employees (N): ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("\n--- Entering details for Employee %d of %d ---", i + 1, n);
        insertEnd();
    }

    printf("\n>>> Successfully created DLL with %d employees! <<<\n", n);
}

// Operation d & e: Delete from Front
void deleteFront() {
    if (head == NULL) {
        printf("\n>>> DLL is EMPTY! Nothing to delete. <<<\n");
        return;
    }

    struct Employee *temp = head;
    head = head->next;

    if (head != NULL) {
        head->prev = NULL;
    }

    printf("\n>>> Deleted Employee - SSN: %s, Name: %s <<<\n", temp->ssn, temp->name);
    free(temp);
}
```

```
}

// Operation c & e: Delete from End

void deleteEnd() {
    if (head == NULL) {
        printf("\n>>> DLL is EMPTY! Nothing to delete. <<<\n");
        return;
    }

    struct Employee *temp = head;

    if (temp->next == NULL) { // Only one node in list
        head = NULL;
    } else {
        while (temp->next != NULL) {
            temp = temp->next;
        }

        temp->prev->next = NULL;
    }

    printf("\n>>> Deleted Employee - SSN: %s, Name: %s <<<\n", temp->ssn, temp->name);
    free(temp);
}

// Operation b: Display DLL status and count total nodes

void display() {
    if (head == NULL) {
        printf("\n>>> DLL Status: EMPTY (0 Nodes) <<<\n");
        return;
    }

    struct Employee *temp = head;
```

```

int count = 0;

printf("\n===== EMPLOYEE DLL RECORDS
=====\\n");

printf("%-5s %-12s %-20s %-10s %-15s %-10s %-12s\\n", "Sl.No", "SSN", "Name", "Dept",
"Designation", "Salary", "Phone No");

printf("-----\\n");

while (temp != NULL) {

    count++;

    printf("%-5d %-12s %-20s %-10s %-15s %-10.2f %-12s\\n",

        count, temp->ssn, temp->name, temp->dept, temp->desig, temp->sal, temp->phno);

    temp = temp->next;

}

printf("=====\\n");

printf("DLL Status: ACTIVE | Total Nodes (Employee Count): %d\\n", count);

}

// Operation e: Demonstrate Double Ended Queue (Deque) operations

void dequeDemo() {

    int dequeChoice;

    while (1) {

        printf("\\n-----\\n");

        printf("  DOUBLE ENDED QUEUE (DEQUE) DEMO MENU  \\n");

        printf("-----\\n");

        printf("1. Push Front (Insert at Head)\\n");

        printf("2. Push Back  (Insert at Tail)\\n");

        printf("3. Pop Front  (Delete from Head)\\n");

        printf("4. Pop Back   (Delete from Tail)\\n");

```

```
printf("5. Display Deque Status\n");
printf("6. Back to Main Menu\n");
printf("-----\n");
printf("Enter Deque choice (1-6): ");
scanf("%d", &dequeChoice);
switch (dequeChoice) {
    case 1: insertFront(); break;
    case 2: insertEnd(); break;
    case 3: deleteFront(); break;
    case 4: deleteEnd(); break;
    case 5: display(); break;
    case 6: return; // Return to main menu
    default: printf("\nInvalid Choice!\n");
}
}
}

// Main Function with Menu Interface
int main() {
    int choice;
    while (1) {
        printf("\n===== \n");
        printf("  DOUBLY LINKED LIST (DLL) MENU  \n");
        printf("===== \n");
        printf("1. Create DLL of N Employees (End Insertion)\n");
        printf("2. Display DLL Status & Node Count\n");
        printf("3. Insert Employee at Front\n");
```

```
printf("4. Delete Employee from Front\n");
printf("5. Insert Employee at End\n");
printf("6. Delete Employee from End\n");
printf("7. Demonstrate DLL as Double Ended Queue (Deque)\n");
printf("8. Exit\n");
printf("-----\n");
printf("Enter your choice (1-8): ");
scanf("%d", &choice);
switch (choice) {
    case 1: createDLL(); break;
    case 2: display(); break;
    case 3: insertFront(); break;
    case 4: deleteFront(); break;
    case 5: insertEnd(); break;
    case 6: deleteEnd(); break;
    case 7: dequeDemo(); break;
    case 8:
        printf("\nExiting Program. Freeing allocated memory... Goodbye!\n");
        while (head != NULL) {
            struct Employee *temp = head;
            head = head->next;
            free(temp);
        }
        exit(0);
    default:
        printf("\nInvalid Choice! Please select between 1 and 8.\n");
}
```

```
    }  
}  
  
return 0;  
  
}
```

OUTPUT:

=====

DOUBLY LINKED LIST (DLL) MENU

=====

1. Create DLL of N Employees (End Insertion)
2. Display DLL Status & Node Count
3. Insert Employee at Front
4. Delete Employee from Front
5. Insert Employee at End
6. Delete Employee from End
7. Demonstrate DLL as Double Ended Queue (Deque)
8. Exit

Enter your choice (1-8):

Enter number of employees (N): 2

--- Entering details for Employee 1 of 2 ---

Enter SSN: 01

Enter Name: MANJU

Enter Department: CSE

Enter Designation: ANALYST

Enter Salary: 50,000

Enter Phone Number:

Department of CSE(ICBT)

>>> Employee record inserted successfully at END! <<<

--- Entering details for Employee 2 of 2 ---

Enter SSN: 02

Enter Name: RAJINI M

Enter Department: ISE

Enter Designation: DEVELOPMENT

Enter Salary: 70000

Enter Phone Number:

9054345667

>>> Employee record inserted successfully at END! <<<

>>> Successfully created DLL with 2 employees! <<<

=====

DOUBLY LINKED LIST (DLL) MENU

=====

1. Create DLL of N Employees (End Insertion)
2. Display DLL Status & Node Count
3. Insert Employee at Front
4. Delete Employee from Front
5. Insert Employee at End
6. Delete Employee from End
7. Demonstrate DLL as Double Ended Queue (Deque)
8. Exit

Enter your choice (1-8): 2

===== EMPLOYEE DLL RECORDS

=====

| Sl.No | SSN | Name | Dept | Designation | Salary | Phone No |
|-------|-----|----------|------|-------------|----------|------------|
| 1 | 01 | MANJU | CSE | ANALYST | 50.00 | ,000 |
| 2 | 02 | RAJINI M | ISE | DEVELOPMENT | 70000.00 | 9054345667 |

DLL Status: ACTIVE | Total Nodes (Employee Count): 2

DOUBLY LINKED LIST (DLL) MENU

1. Create DLL of N Employees (End Insertion)
2. Display DLL Status & Node Count
3. Insert Employee at Front
4. Delete Employee from Front
5. Insert Employee at End
6. Delete Employee from End
7. Demonstrate DLL as Double Ended Queue (Deque)
8. Exit

Enter your choice (1-8): 7

DOUBLE ENDED QUEUE (DEQUE) DEMO MENU

1. Push Front (Insert at Head)
2. Push Back (Insert at Tail)
3. Pop Front (Delete from Head)

6. Back to Main Menu

Enter Deque choice (1-6): 1

Enter SSN: 01

Enter Name: MANJU

Enter Department: CSE

Enter Designation: ANALYST

Enter Salary: 80000

Enter Phone Number: 098788989

>>> Employee record inserted successfully at FRONT! <<<

DOUBLE ENDED QUEUE (DEQUE) DEMO MENU

1. Push Front (Insert at Head)

2. Push Back (Insert at Tail)

3. Pop Front (Delete from Head)

4. Pop Back (Delete from Tail)

5. Display Deque Status

6. Back to Main Menu

Enter Deque choice (1-6): 6

=====

DOUBLY LINKED LIST (DLL) MENU

=====

1. Create DLL of N Employees (End Insertion)

2. Display DLL Status & Node Count

3. Insert Employee at Front
4. Delete Employee from Front
5. Insert Employee at End
6. Delete Employee from End
7. Demonstrate DLL as Double Ended Queue (Deque)
8. Exit

Enter your choice (1-8): 8

Exiting Program. Freeing allocated memory... Goodbye!

=== Code Execution Successful ===

Exp. No. 5

Develop a menu driven Program in C for the following operations on Binary Search Tree (BST) of Integers.

a. Create a BST of N Integers

b. Traverse the BST in Inorder, Preorder and Post Order

c. Search the BST for a given element (KEY) and report the appropriate message

d. Exit

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Structure for BST Node
```

```
struct Node {
```

```
    int data;
```

```
    struct Node *left;
```

```
    struct Node *right;
```

```
};
```

// Function to create a new BST node

```
struct Node* createNode(int value) {  
    struct Node *newNode = (struct Node*) malloc(sizeof(struct Node));  
    if (newNode == NULL) {  
        printf("\nMemory allocation failed!\n");  
        return NULL;  
    }  
    newNode->data = value;  
    newNode->left = NULL;  
    newNode->right = NULL;  
    return newNode;  
}
```

// Function to insert an integer element into the BST

```
struct Node* insert(struct Node *root, int value) {  
    if (root == NULL) {  
        return createNode(value);  
    }  
    if (value < root->data) {  
        root->left = insert(root->left, value);  
    } else if (value > root->data) {  
        root->right = insert(root->right, value);  
    } else {  
        printf("\nDuplicate key (%d) ignored! BST values must be distinct.\n", value);  
    }  
    return root;  
}
```

// Operation a: Create a BST of N Integers

```
struct Node* createBST(struct Node *root) {  
    int n, value, i;  
    printf("\nEnter total number of integers (N): ");  
    scanf("%d", &n);  
    for (i = 0; i < n; i++) {  
        printf("Enter integer element %d of %d: ", i + 1, n);  
        scanf("%d", &value);  
        root = insert(root, value);  
    }  
    printf("\n>>> BST successfully created with %d elements! <<<\n", n);  
    return root;  
}
```

// Operation b: Inorder Traversal (Left -> Root -> Right)

```
void inorder(struct Node *root) {  
    if (root != NULL) {  
        inorder(root->left);  
        printf("%d ", root->data);  
        inorder(root->right);  
    }  
}
```

// Operation b: Preorder Traversal (Root -> Left -> Right)

```
void preorder(struct Node *root) {  
    if (root != NULL) {  
        printf("%d ", root->data);  
        preorder(root->left);  
        preorder(root->right);  
    }  
}
```

```
}  
  
}  
  
// Operation b: Postorder Traversal (Left -> Right -> Root)  
  
void postorder(struct Node *root) {  
    if (root != NULL) {  
        postorder(root->left);  
        postorder(root->right);  
        printf("%d ", root->data);  
    }  
}  
  
// Helper function to display all traversals  
  
void displayTraversals(struct Node *root) {  
    if (root == NULL) {  
        printf("\n>>> BST is EMPTY! Please create or insert nodes first. <<<\n");  
        return;  
    }  
  
    printf("\n--- BST TRAVERSALS ---\n");  
  
    printf("Inorder Traversal  :");  
  
    inorder(root);  
  
    printf("\nPreorder Traversal  :");  
  
    preorder(root);  
  
    printf("\nPostorder Traversal :");  
  
    postorder(root);  
  
    printf("\n");  
}  
  
  
// Operation c: Search the BST for a given element (KEY)
```

```
void search(struct Node *root, int key) {

    if (root == NULL) {

        printf("\n>>> Key %d NOT FOUND in the BST! (Tree is empty or key doesn't exist) <<<\n", key);

        return;

    }

    if (key == root->data) {

        printf("\n>>> SUCCESS: Key %d FOUND in the BST! <<<\n", key);

        return;

    } else if (key < root->data) {

        search(root->left, key);

    } else {

        search(root->right, key);

    }

}

// Helper function to free allocated memory recursively on exit

void freeTree(struct Node *root) {

    if (root != NULL) {

        freeTree(root->left);

        freeTree(root->right);

        free(root);

    }

}

// Main Function with Menu Interface

int main() {

    struct Node *root = NULL;

    int choice, value, key;

    while (1) {
```

```
printf("\n=====\\n");

printf("  BINARY SEARCH TREE (BST) MENU  \\n");

printf("=====\\n");

printf("1. Create / Insert N Integers into BST\\n");

printf("2. Traverse BST (Inorder, Preorder, Postorder)\\n");

printf("3. Search for a KEY Element\\n");

printf("4. Exit\\n");

printf("-----\\n");

printf("Enter your choice (1-4): ");

scanf("%d", &choice)

switch (choice) {

    case 1:

        root = createBST(root);

        break;

    case 2:

        displayTraversals(root);

        break;

    case 3:

        if (root == NULL) {

            printf("\\n>>> BST is EMPTY! Please create or insert nodes first. <<<\\n");

        } else {

            printf("\\nEnter KEY element to search: ");

            scanf("%d", &key);

            search(root, key);

        }

        break;
```

```
        case 4:

            printf("\nExiting BST Program. Freeing allocated memory... Goodbye!\n");

            freeTree(root);

            exit(0);

        default:

            printf("\nInvalid choice! Please select between 1 and 4.\n");

    }

}

return 0;

}
```

OUTPUT:

```
=====

    BINARY SEARCH TREE (BST) MENU

=====

1. Create / Insert N Integers into BST
2. Traverse BST (Inorder, Preorder, Postorder)
3. Search for a KEY Element
4. Exit

-----

Enter your choice (1-4): 1

Enter total number of integers (N): 2

Enter integer element 1 of 2: 5

Enter integer element 2 of 2: 2

>>> BST successfully created with 2 elements! <<<
```

BINARY SEARCH TREE (BST) MENU

=====

1. Create / Insert N Integers into BST
2. Traverse BST (Inorder, Preorder, Postorder)
3. Search for a KEY Element
4. Exit

Enter your choice (1-4): 2

--- BST TRAVERSALS ---

Inorder Traversal : 2 5

Preorder Traversal : 5 2

Postorder Traversal : 2 5

=====

BINARY SEARCH TREE (BST) MENU

=====

1. Create / Insert N Integers into BST
2. Traverse BST (Inorder, Preorder, Postorder)
3. Search for a KEY Element
4. Exit

Enter your choice (1-4): 3

Enter KEY element to search: 5

>>> SUCCESS: Key 5 FOUND in the BST! <<<

=====

BINARY SEARCH TREE (BST) MENU

1. Create / Insert N Integers into BST
2. Traverse BST (Inorder, Preorder, Postorder)
3. Search for a KEY Element
4. Exit

Enter your choice (1-4): 4

Exiting BST Program. Freeing allocated memory... Goodbye!

=== Code Execution Successful ===

Exp. No. 6

Develop a Program in C for an Employee Record Management System using Hashing with the following operations:

- a. Create a hash table to store employee records using Employee ID as the key.
- b. Insert employee records into the hash table using the Hash function $H: K \rightarrow L$ as $H(K) = K \bmod m$
- c. Resolve collisions using Linear Probing.
- d. Search for an employee record using a given Employee ID and display the details.
- e. Display the contents of the hash table along with the corresponding memory locations.

CODE:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#define TABLE_SIZE 10 // Hash table size (m = 10)

// Structure for Employee Record

struct Employee {

    int empID;

    char name[50];

    char dept[30];
```

```
float salary;

    int isOccupied; // 0 = Empty slot, 1 = Occupied slot
};

// Global Hash Table

struct Employee hashTable[TABLE_SIZE];

// Helper function to safely read string inputs replacing gets()

void readString(char *str, int size) {

    fgets(str, size, stdin);

    size_t len = strlen(str);

    if (len > 0 && str[len - 1] == '\n') {

        str[len - 1] = '\0';

    }

}

// Function a: Initialize the Hash Table

void initHashTable() {

    for (int i = 0; i < TABLE_SIZE; i++) {

        hashTable[i].empID = -1;

        hashTable[i].isOccupied = 0; // Mark slot as empty

    }

}

// Function b & c: Insert Employee Record with Hash Function  $H(K) = K \bmod m$  and Linear Probing

void insertEmployee() {

    int id, index, originalIndex;

    struct Employee emp;

    printf("\nEnter Employee ID (4-digit Integer Key K): ");
```

```
scanf("%d", &id);

// Hash Function:  $H(K) = K \bmod m$ 

index = id % TABLE_SIZE;

originalIndex = index;

// Linear Probing for Collision Resolution

while (hashTable[index].isOccupied == 1) {

    // If employee with same ID already exists

    if (hashTable[index].empID == id) {

        printf("\n>>> ERROR: Employee ID %d already exists in the Hash Table! <<<\n", id);

        return;

    }

    printf("Collision occurred at Index %d for Key %d! Applying Linear Probing...\n", index, id);

    index = (index + 1) % TABLE_SIZE;

    // Table is completely full

    if (index == originalIndex) {

        printf("\n>>> HASH TABLE OVERFLOW! Hash table is full. Cannot insert Employee ID %d. <<<\n", id);

        return;

    }

}

// Read Employee Details

emp.empID = id;

printf("Enter Employee Name: ");

getchar(); // Clear leftover newline

readString(emp.name, sizeof(emp.name));

printf("Enter Department: ");
```

```
readString(emp.dept, sizeof(emp.dept));

printf("Enter Salary: ");

scanf("%f", &emp.salary);

emp.isOccupied = 1;

// Place employee at resolved index

hashTable[index] = emp;

printf("\n>>> Success: Employee ID %d stored at Hash Index %d! <<<\n", id, index);

}

// Function d: Search for an Employee Record by Employee ID

void searchEmployee() {

    int id, index, originalIndex;

    printf("\nEnter Employee ID to search: ");

    scanf("%d", &id);

    index = id % TABLE_SIZE;

    originalIndex = index;

    while (hashTable[index].isOccupied == 1 || hashTable[index].empID != -1) {

        if (hashTable[index].isOccupied == 1 && hashTable[index].empID == id) {

            printf("\n===== EMPLOYEE DETAILS FOUND\n");

            printf("Memory Location / Index : %d (Address: %p)\n", index,

(void*)&hashTable[index]);

            printf("Employee ID      : %d\n", hashTable[index].empID);

            printf("Name          : %s\n", hashTable[index].name);

            printf("Department    : %s\n", hashTable[index].dept);

            printf("Salary        : %.2f\n", hashTable[index].salary);

            printf("===== \n");

        }

    }

};
```

```

    index = (index + 1) % TABLE_SIZE;

    // Traversed entire table and looped back

    if (index == originalIndex) {
        break;
    }
}

printf("\n>>> Employee ID %d NOT FOUND in the Hash Table. <<<\n", id);
}

// Function e: Display contents of Hash Table with memory locations

void displayHashTable() {

    printf("\n===== HASH TABLE CONTENTS
=====\\n");

    printf("%-12s %-18s %-12s %-20s %-15s %-10s\\n", "Index", "Memory Address", "Emp ID",
"Name", "Department", "Salary");

    printf("-----\\n");

    for (int i = 0; i < TABLE_SIZE; i++) {

        if (hashTable[i].isOccupied == 1) {

            printf("%-12d %-18p %-12d %-20s %-15s %-10.2f\\n",

                i, (void*)&hashTable[i], hashTable[i].empID, hashTable[i].name, hashTable[i].dept,
hashTable[i].salary);

        } else {

            printf("%-12d %-18p %-12s %-20s %-15s %-10s\\n",

                i, (void*)&hashTable[i], "---", "[EMPTY SLOT]", "---", "---");

        }

    }

    printf("=====
=====\\n");

```

```
}

// Main Function with Menu Interface

int main() {

    int choice;

    initHashTable(); // Initialize hash table slots

    while (1) {

        printf("\n=====\\n");

        printf("  EMPLOYEE RECORD HASHING SYSTEM MENU  \\n");

        printf("=====\\n");

        printf("1. Insert Employee Record\\n");

        printf("2. Search Employee Record by ID\\n");

        printf("3. Display Hash Table & Memory Locations\\n");

        printf("4. Exit\\n");

        printf("-----\\n");

        printf("Enter your choice (1-4): ");

        scanf("%d", &choice);

        switch (choice) {

            case 1:

                insertEmployee();

                break;

            case 2:

                searchEmployee();

                break;

            case 3:

                displayHashTable();

                break;
```

```
        case 4:

            printf("\nExiting Program. Goodbye!\n");

            exit(0);

        default:

            printf("\nInvalid Choice! Please enter a number between 1 and 4.\n");

    }

}

return 0;

}
```

OUTPUT:

```
=====

EMPLOYEE RECORD HASHING SYSTEM MENU

=====

1. Insert Employee Record
2. Search Employee Record by ID
3. Display Hash Table & Memory Locations
4. Exit

-----

Enter your choice (1-4): 1

Enter Employee ID (4-digit Integer Key K): 1254

Enter Employee Name: MANJUNATHA

Enter Department: CSE

Enter Salary: 67000

>>> Success: Employee ID 1254 stored at Hash Index 4! <<<
```

=====

EMPLOYEE RECORD HASHING SYSTEM MENU

=====

1. Insert Employee Record
2. Search Employee Record by ID
3. Display Hash Table & Memory Locations
4. Exit

Enter your choice (1-4): 2

Enter Employee ID to search: 1

>>> Employee ID 1 NOT FOUND in the Hash Table. <<<

=====

EMPLOYEE RECORD HASHING SYSTEM MENU

=====

1. Insert Employee Record
2. Search Employee Record by ID
3. Display Hash Table & Memory Locations
4. Exit

Enter your choice (1-4): 2

Enter Employee ID to search: 1254

===== EMPLOYEE DETAILS FOUND =====

Memory Location / Index : 4 (Address: 0x4041f0)

Employee ID : 1254

Name : MANJUNATHA

Department : CSE

Salary : 67000.00

=====

=====

EMPLOYEE RECORD HASHING SYSTEM MENU

=====

1. Insert Employee Record
2. Search Employee Record by ID
3. Display Hash Table & Memory Locations
4. Exit

Enter your choice (1-4): 3

===== HASH TABLE CONTENTS

=====

| Index | Memory Address | Emp ID | Name | Department | Salary |
|-------|----------------|--------|------|------------|--------|
|-------|----------------|--------|------|------------|--------|

| | | | | | |
|---|----------|------|--------------|-----|----------|
| 0 | 0x404080 | --- | [EMPTY SLOT] | --- | --- |
| 1 | 0x4040dc | --- | [EMPTY SLOT] | --- | --- |
| 2 | 0x404138 | --- | [EMPTY SLOT] | --- | --- |
| 3 | 0x404194 | --- | [EMPTY SLOT] | --- | --- |
| 4 | 0x4041f0 | 1254 | MANJUNATHA | CSE | 67000.00 |
| 5 | 0x40424c | --- | [EMPTY SLOT] | --- | --- |
| 6 | 0x4042a8 | --- | [EMPTY SLOT] | --- | --- |
| 7 | 0x404304 | --- | [EMPTY SLOT] | --- | --- |
| 8 | 0x404360 | --- | [EMPTY SLOT] | --- | --- |
| 9 | 0x4043bc | --- | [EMPTY SLOT] | --- | --- |

EMPLOYEE RECORD HASHING SYSTEM MENU

=====

1. Insert Employee Record
 2. Search Employee Record by ID
 3. Display Hash Table & Memory Locations
 4. Exit
-

Enter your choice (1-4): 4

Exiting Program. Goodbye!

=== Code Execution Successful ===